GitHub Copilot 워크플로 과정

GitHub Copilot 공통 정리

실행 모드 · 환경별 차이 · 토큰과 비용 · 모델 선택 + 실습

1 실행 모드

2 환경별 차이 + 실습

3 토큰·비용 구조

4 모델 선택 + 실습

01

Copilot 실행 모드

Interactive · Plan · Autopilot 세 가지 모드와 권한 수준을 정리합니다.

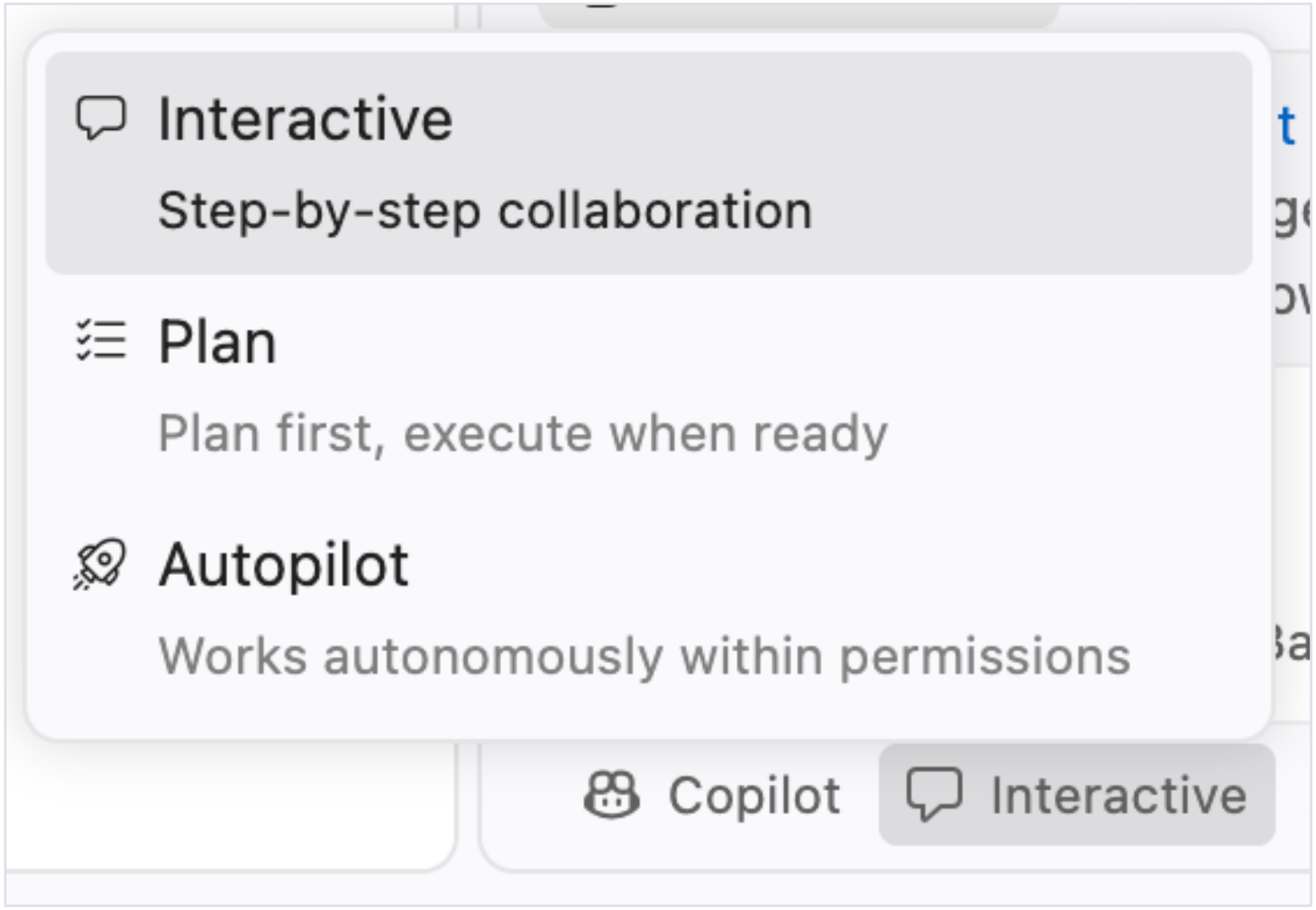
세션 모드 3가지

VS Code · Copilot CLI · GitHub Copilot 앱 공통

모드	동작	이럴 때
Interactive	한 단계씩 협업, 필요 시 도구 승인 요청	코드 이해, 함께 하는 구현 (기본)
Plan	코드 작성 전 조사·계획 작성 → 승인 후 실행	크거나 모호한 작업
Autopilot	첫 지시 후 입력 없이 끝까지 진행	테스트 작성·리팩터링·CI 수정 등 **목표가 분명한** 작업

전환 — VS Code: 입력창 아래 모드 버튼 · CLI: `Shift+Tab` · 앱: 모드 버튼, `/plan`

VS Code 모드 선택 화면



1 **입력창 아래 모드 버튼**

`Interactive` 표시 버튼 클릭

2 **세 모드 중 선택**

Interactive · Plan · Autopilot

3 **모드마다 한 줄 설명**

한 단계씩 협업 · 계획 먼저 · 자율 진행

하네스에 따라 다른 모드 구성

Copilot 하네스 (기본)

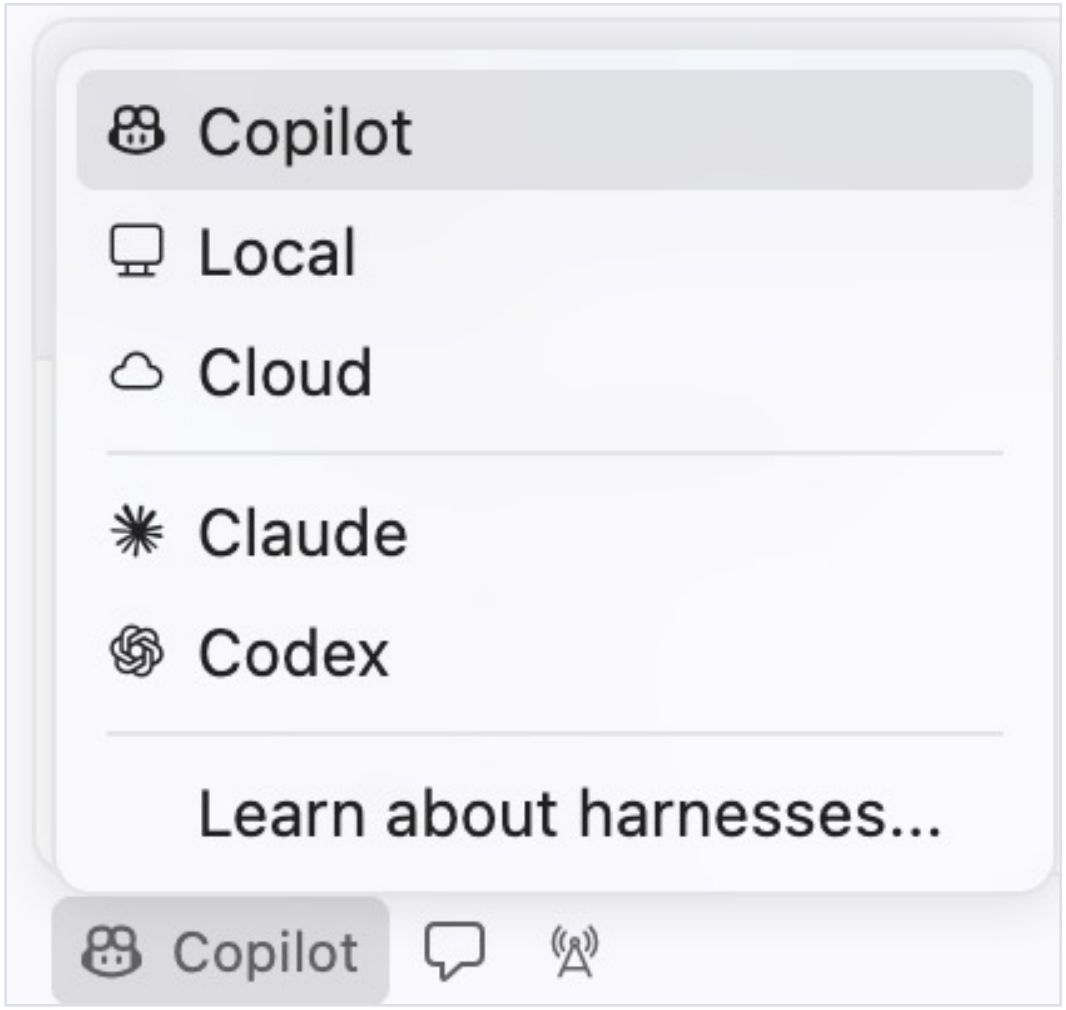
- Copilot CLI 기반, 새 기본값
- 모드: Interactive · Plan · Autopilot
- 에이전트: Agent + 커스텀 에이전트
- 권한: Manual · Assisted · Allow all
- 첫 실행 시 새 Copilot experience 안내

Local 하네스 (예전 방식)

- 모드 버튼 없음
- 에이전트: Agent · Ask · Plan + 커스텀
- 권한: Default · Allow all · Autopilot
- GitHub Docs IDE 문서의 Ask·Agent·Plan
- Autopilot 권한은 Preview

강의 기준: **Copilot 하네스 + Interactive + Manual permissions**

하네스는 새 세션에서 고르기



1 **새 세션의 `Copilot` 버튼**

입력창 아래 → 메뉴에서 Copilot · Local · Cloud …

2 **진행 중인 대화에선 변경 불가**

입력창 아래엔 모드 · 권한만 표시

3 **바꾸려면 새 세션부터**

새 세션 → 하네스 선택 → 질문

VS Code 권한(Permission) 수준

권한 수준	동작
Manual (Local: Default)	승인 설정에 해당하지 않는 도구는 실행 전 확인 (기본)
Assisted (Experimental)	AI 판정기가 도구 호출 위험 평가, 통과 못한 것만 확인
Allow all	모든 도구 호출을 묻지 않고 실행
Autopilot (Local, Preview)	자동 승인 + 작업이 끝날 때까지 계속 진행
Sandboxing for terminal	터미널 명령을 파일 시스템·네트워크가 제한된 샌드박스에서 실행

조직 정책으로 자동 승인이 막혀 있으면 Manual/Default만 선택 가능

02

환경별 차이

VS Code · Copilot CLI · GitHub Copilot 앱에서 같은 기능을 찾는 방법을 정리합니다.

세 환경 한눈에 비교

모드는 세 환경 모두 Interactive · Plan · Autopilot

에디터

VS Code

보면서 묻고 고치기, 인라인 자동완성. 실습 기본 환경

터미널

Copilot CLI

키보드만으로 빠르게 작업. SSH 서버, 스크립트화

데스크톱 앱

GitHub Copilot 앱

병렬 세션으로 이슈 여러 개 동시 처리. 세션별 PR

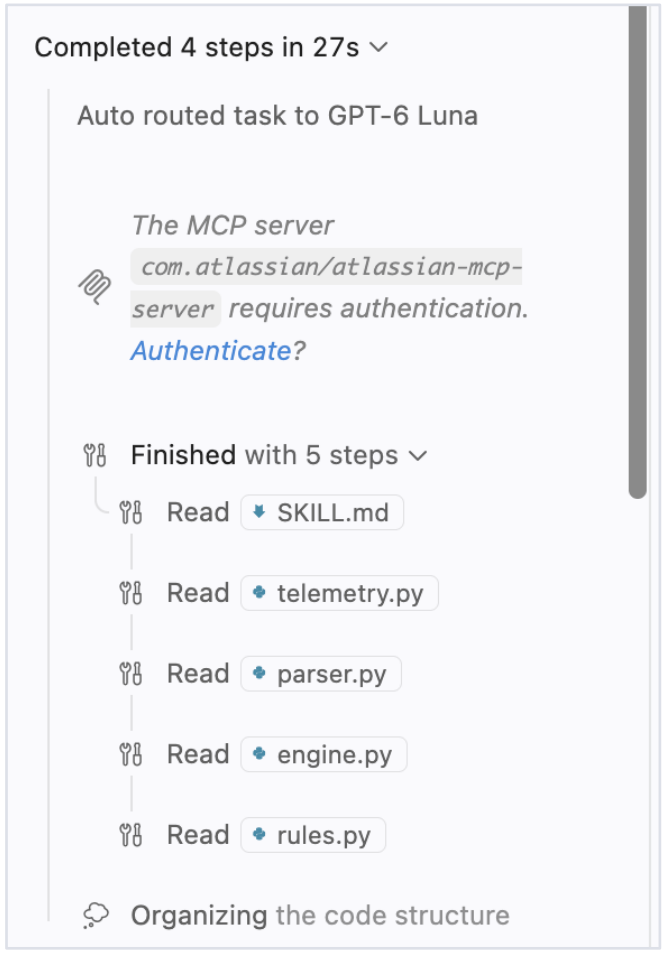
Instructions · Skill · 커스텀 에이전트 · MCP · Auto 모델은 세 환경 모두 지원

같은 기능, 다른 위치

하고 싶은 일	VS Code	Copilot CLI	Copilot 앱
새 대화/세션	채팅 상단 `+`	`/clear`	프로젝트 옆 `+`
모드 전환	입력창 아래 모드 버튼	`Shift+Tab`	모드 버튼, `/plan`
모델 선택	입력창 안 `Auto`	`/model`	모델 메뉴, `/model`
파일 첨부	`#file:경로`	`@파일경로`	`@파일`
리뷰	Source Control 뷰	`/review`	`/review`
사용량 보기	상태 표시줄 Copilot 아이콘	`/usage`	`/usage`
작업 중단	채팅의 ■ Stop	`Ctrl+C`	정지 버튼

응답 근거 확인 — Completed N steps

Copilot이 무엇을 읽고 답했는지 항상 확인합니다.



1 단계 펼치기

맨 위 `Completed N steps` → 안의 `Finished with N steps`

2 모델 확인

`Auto routed task to <모델>` — Auto가 고른 모델

3 읽은 파일 확인

`Read <파일>` 줄. Skill은 `SKILL.md` 부터 읽음

실습

Skill로 코드 흐름 파악

`/explain-flow` Skill을 실행하고, Completed N steps 에서 Copilot이 읽은 파일을 확인합니다.

준비

- VS Code로 repo 루트 폴더 열기
- 새 세션: 하네스 `Copilot` · Interactive · Manual permissions

실습 1

/explain-flow 실행 + 읽은 파일 확인

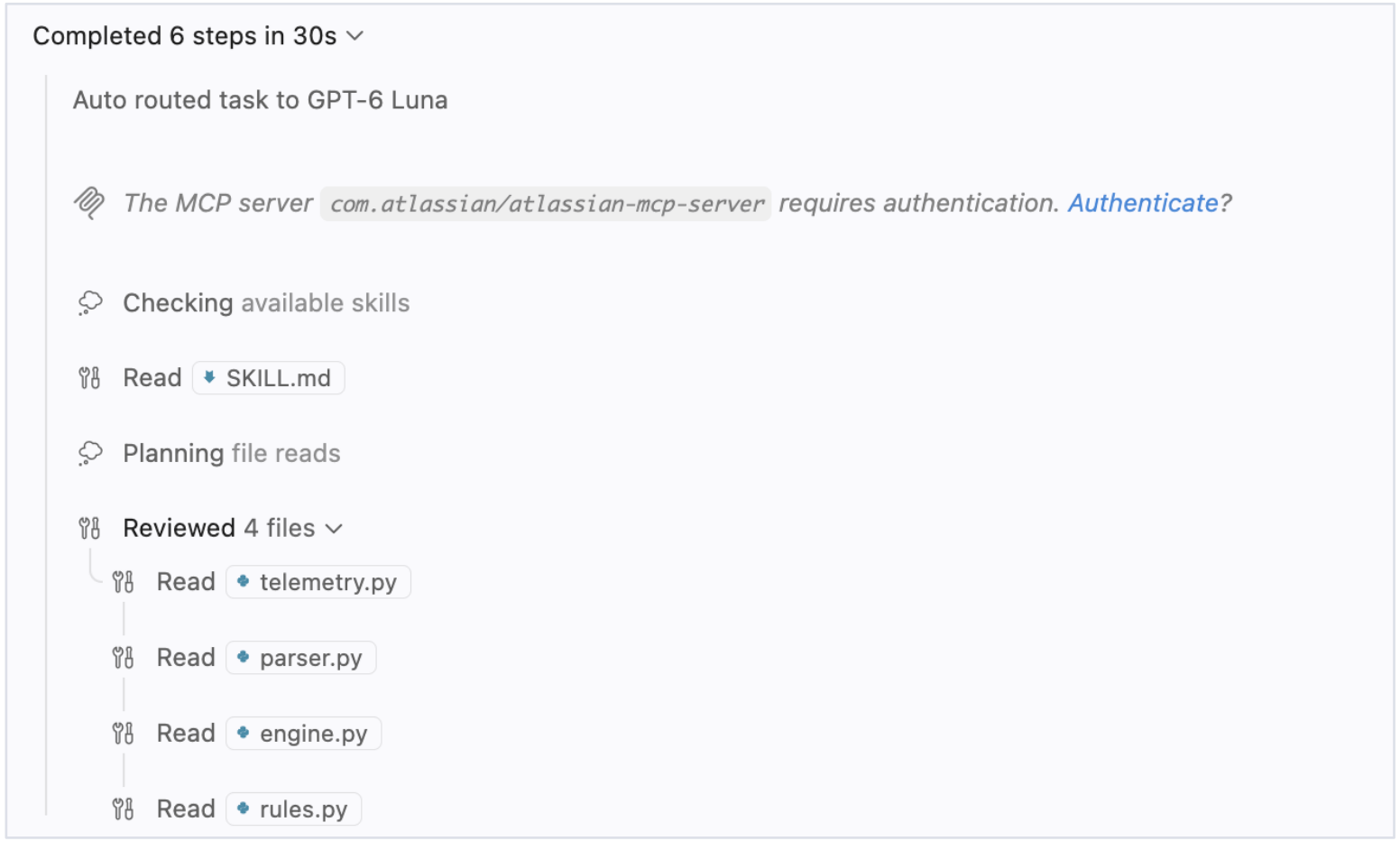
1. 새 세션: 하네스 **Copilot · Interactive · Manual permissions**
2. `/explain` 입력 → **/explain-flow** 선택 → Enter
3. 응답: `POST /telemetry` → `store.add_alarm` 호출 경로 확인
4. **Completed N steps** → **Reviewed 4 files** 펼치기

채팅 입력

```
/explain-flow
```

✅ 파일:함수 순서의 호출 경로 ✅ SKILL.md 다음 4개 파일만 읽음

Completed N steps — 읽은 파일 4개



1 **Auto routed task to …**

Auto가 고른 모델

2 **Read SKILL.md**

Skill 본문을 먼저 읽음

3 **Reviewed 4 files**

`telemetry` · `parser` · `engine` · `rules` 만

03

토큰과 비용 구조

한 요청에 무엇이 실리는지, AI Credits가 어떻게 과금되는지 정리합니다.

"한 요청"에 실리는 것 = 전부 토큰

질문 한 줄만 보내는 것 같지만, 아래가 매번 함께 전송됩니다.

```
[내가 보낸 요청 1건]
 + .github/copilot-instructions.md   # 항상
 + 경로가 맞는 *.instructions.md     # 해당 파일
 + #file / #codebase 결과            # 내가 첨부
 + 열린 편집기 · 선택 영역           # 자동 포함
 + 도구 호출 결과                    # Agent가 생성
 + 이 대화의 히스토리 전체           # 매 턴 재전송
 = 프롬프트 크기 → 비용 + 응답 품질
```

토큰이 늘어나는 네 가지 원리

고정 비용

Instructions

40줄과 200줄은 매 요청 5배 차이 → 짧게, 프로젝트 고유 규칙만

복리

히스토리

대화가 길수록 매 턴 전부 재전송 → 주제가 바뀌면 **새 대화(+)**

곱셈

도구 호출

파일 20개를 열면 컨텍스트 20번 증가 → `#file` 로 **직접 지정**

품질

큰 컨텍스트

무관한 정보가 많을수록 핵심 놓침 → "맞는 것만" 골라 넣기

과금 단위: AI Credits (2026-06 이후)

$0.01

1 AI Credit

1,900

Business 좌석당 월 포함 credits

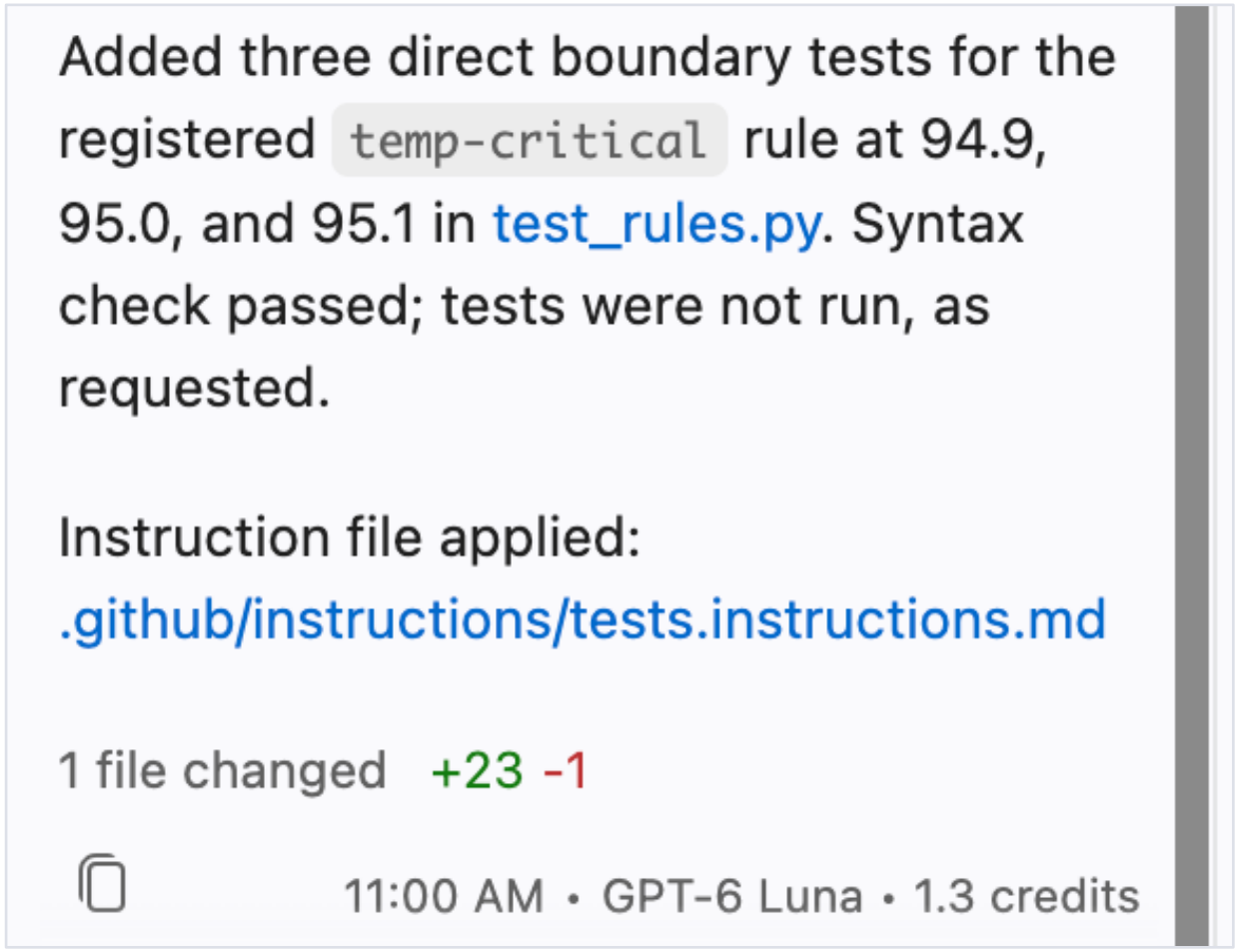
10%

Auto 사용 시 모델 비용 할인

- 조직·엔터프라이즈 **공용 풀**에 합산 (100석 = 190,000 credits)
- 매월 1일 00:00 UTC 초기화, 이월 없음
- 소비량 = **모델** × **토큰**(컨텍스트·출력)
- 인라인 자동완성은 유료 플랜에서 별도 소비 없음

사용량 확인 위치

매일 볼 두 곳만 먼저 익힙니다.



1 **응답 맨 아래**

요청 1건의 모델과 `N credits`

2 **상태 표시줄 Copilot 아이콘**

이번 달 사용률: `Credits N% used`

3 **legacy 주의**

`premium requests` 방식은 기존 연간 Pro/Pro+ 사용자에게만 남음

04

모델 선택

Auto의 동작 원리와 직접 모델을 지정할 경우를 정리합니다.

기본은 Auto, 직접 지정은 3가지 경우

경우	선택	이유
대부분의 구현·수정·질문	**Auto**	난이도별 자동 선택 + 10% 할인 + 사용량 예측 가능
대규모 설계 — 구조 변경, Plan 모드	**상위(추론형) 모델**	한 번에 끝내는 편이 오히려 저렴
단순 반복 — 패턴 20번, 이름 바꾸기	**경량 모델**	빠르고 저렴, 품질 차이 거의 없음
재현성 — 결과를 비교·기록 (실험, 평가)	**모델 고정**	Auto는 요청마다 다른 모델 가능

"좋은 모델이 안전"이 아니라 **"맞는 모델이 안전"** — 상위 모델은 크레딧을 더 소비

Auto가 모델을 고르는 방식

GitHub Docs Auto model selection

1. **실시간 시스템 상태**
   지금 여유 있고 빠른 모델 (가용성·지연·오류율)

2. **작업 복잡도**
   묻는 언어가 아니라 **무엇을 하려는지**로 판단

3. **대화 중 수시 전환 없음**
   캐시 경계·복잡도 급변 시에만 전환

4. **과금**
   실제 선택된 모델 기준, 유료 플랜 10% 할인

선택에서 제외

플랜에 없는 모델
관리자 정책 차단 모델
데이터 상주/FedRAMP 전용
평가용 모델

실습

같은 질문, 다른 방식

같은 질문을 넓게 / 좁게 보내 Session Info 를 비교하고
사용량 기준선을 기록합니다.

준비

- Auto 모델 · Interactive 모드 유지
- 메모장 (credits 기록용)

실습 2-1

(a) 첨부 없이 넓게 묻기

1. 채팅 상단 + 로 새 대화 — **Interactive · Auto**
2. 오른쪽 프롬프트 전송
3. 입력창 아래 오른쪽 원형 아이콘(예: 17%) → **Session Info**
4. credits · 컨텍스트 토큰 메모

프롬프트 / 명령

```
이 프로젝트의 알람 룰들이 각각 어떤 임계값과
심각도를 쓰는지 표로 정리하고,
새 룰은 어떻게 추가하는지 알려줘.
```

✅ **temp-critical ≥ 95.0 / CRITICAL** ✅ **pressure-high ≥ 3.0 / WARNING**

실습 2-2

(b) 파일 첨부 + 범위 좁혀 묻기

1. `+` 로 새 대화
2. 입력창 왼쪽 아래 **+** → **Files & Folders...** → `rules` → `rules.py`
3. 입력창 위 `rules.py` 칩 확인 후 프롬프트 전송
4. Session Info 비교 + **Copilot 아이콘** → **Credits N% used** 기록

프롬프트 / 명령

```
첨부한 rules.py 만 보고 답해.
다른 파일은 열거나 검색하지 마.
룰 두 개의 임계값과 심각도를 표로 정리하고,
새 룰 추가 방법은 한 줄로.
```

✅ **(a)와 같은 답** ✅ **서브에이전트 없이 빠르게 끝남** ✅ **기준선 credits 기록**

실측 — 비용보다 속도 차이

방식	단계 / 시간	컨텍스트	비용
(a) 첨부 없이 넓게	4 steps + 서브에이전트 6분+	67.1K	0.9 credits
(b) 첨부 + 범위 지정	5 steps / 25s	66.0K	0.9 credits
(c) (b) + Gemini 3.8 Flash 지정	**2 steps / 12s**	66.3K	**5.8 credits**

비용 최적화 1순위: **Auto 유지, 대화 중 모델 바꾸지 않기**

핵심 요약

1. **모드**: Interactive · Plan · Autopilot — 큰 일은 Plan → 검토 → 실행
2. Autopilot은 목표가 분명할 때만
3. 응답 위 **Completed N steps** 펼쳐서 읽은 내용 확인
4. **컨텍스트 = 비용**: instructions는 고정비, 히스토리는 복리, 도구 호출은 곱셈
5. **모델은 Auto가 기본** — 대규모 설계·단순 반복·재현성만 직접 지정